

基于知识注入的大语言模型水印

陈可江, 李帅, 张卫明*, 俞能海
(中国科学技术大学网络空间安全学院, 合肥 230026)

摘要: 大语言模型凭借其出色的文本理解能力和生成能力, 在自然语言处理任务上取得了优异的表现。训练大语言模型需要高质量的标注数据、昂贵的算力资源, 这使其成为重要的数字资产, 具有极高的商业价值, 但是却存在被盗用等知识产权泄露风险。因此, 亟须发展大语言模型水印技术, 以保护模型的版权。现有基于无盒水印的大语言模型水印可以保护模型的版权, 但是这些方法水印隐蔽性不足、生成文本质量下降, 并且难以应用于模型开源场景。为了解决上述问题, 提出了一种基于知识注入的大语言模型水印方法。在嵌入水印阶段, 将水印嵌入到自定义知识中, 并通过监督微调让大语言模型学会带水印的知识。在水印提取阶段, 模型所有者只需要设计与水印知识相关的问题, 询问待检测的大语言模型, 根据模型的回答提取水印信息, 并通过实验验证了该方法的有效性、保真性和鲁棒性。

关键词: 大语言模型; 水印; 知识注入; 监督微调; 隐蔽; 鲁棒性

中图分类号: TP309 **文献标识码:** A

DOI: 10.20172/j.issn.2097-3136.240105

Watermarking for large language models based on knowledge injection

CHEN Kejiang, LI Shuai, ZHANG Weiming*, YU Nenghai
(Department of Cyber Science and Technology, University of Science and Technology of China, Hefei 230026, China)

Abstract: Large language models have shown outstanding performance on natural language processing tasks due to their exceptional text understanding and generation capabilities. Training large language models demands high-quality annotated data and expensive computational resources, making them significant digital assets with considerable commercial value while susceptible to intellectual property theft. Therefore, Developing watermarking technologies is key to ensuring copyright protection for large language models. Existing large language models watermarking methods based on box-free watermarks that provide strong copyright safeguards. However, these methods frequently suffer from inadequate concealment, reduced text quality, and difficulties in implementation within open-source environments. To address the above issues, a large language models watermarking method based on knowledge injection was proposed. During the watermark embedding phase, the watermark was embedded into custom knowledge and the model learns the watermarked knowledge through supervised fine-tuning. In the watermark extraction phase, the model owner only needed to design questions related to the watermark knowledge and query the model to be tested, extracting the watermark information based on the model's responses. The experiments conduct validate the effectiveness, fidelity and robustness of the proposed method.

Keywords: large language models; watermark; knowledge injection; supervised finetune; concealment; robustness

0 引言

大语言模型 (Large Language Models, LLMs) 如 GPT-4^[1]、ChatGLM^[2] 和 LLaMA 2^[3] 等, 凭借其优秀的文本理解能力, 在情感分析、文本生成、机器翻译等自然语言处理任务上取得了优异的表现。模

型所有者可以通过利用大语言模型的优秀性能为客户需求的下游任务提供服务来创造商业价值, 例如 OpenAI 的 ChatGPT 已经成为最为火爆的商用聊天机器人之一。然而大语言模型在为用户提供功能服务的同时, 不可避免地存在版权泄露的风险。对于一

基金项目: 国家自然科学基金 (U2336206, 62102386, 62072421)

引用格式: 陈可江, 李帅, 张卫明, 等. 基于知识注入的大语言模型水印 [J]. 网络空间安全科学学报, 2024, 2 (1): 63 - 71.

Citation Format: CHEN K J, LI S, ZHANG W M, et al. Watermarking for large language models based on knowledge injection[J]. Journal of Cybersecurity, 2024, 2 (1): 63 - 71.

个恶意的用户而言,得到的无论是大语言模型 API (Application Programming Interface, API), 还是完整的大语言模型,他都可以通过倒卖大语言模型功能服务为自己牟利。此外,一些 Github、Hugging Face 上的非商用开源大语言模型,恶意用户也可能私自将这些模型用于商业行为。这些非法使用他人模型的行为严重侵害了大语言模型拥有者的权益。因此,亟须发展大语言模型版权保护技术。

为大语言模型添加水印是保护大语言模型版权的有效方法^[4]。传统的模型水印主要分为白盒水印^[5-6]、无盒水印^[7-8]和黑盒水印^[9-12]。白盒水印方法主要是将水印嵌入到模型的参数中,因此在提取水印阶段需要访问模型的参数信息。然而在大多数应用场景中,很难得到模型的参数信息,因此白盒水印方法并不具有很强的实用性。

相比白盒水印需要访问模型的参数信息,无盒水印是在模型输出中嵌入水印,因此模型拥有者仅需得到模型的输出就可以提取水印。目前,针对大语言模型的大部分水印方法是基于无盒水印的,其中主要包括基于修改 logit^[13-15]和修改 token 采样^[16-17]的方法。基于修改 logit 的水印方法将 token 分为红绿名单,在生成 token 时通过修改 logit 使模型偏向生成绿名单中的 token,在水印检测阶段通过显著性检验检测水印。然而,此方法修改了 logit,会在一定程度上导致生成文本质量下降。修改 token 采样的方法基于 token 采样固有的随机性。在 token 采样阶段,利用伪随机数生成器生成伪随机数序列,以指导 token 采样。通过评估生成文本与伪随机数之间的相关性来检测水印。

然而,以上无盒水印方法主要保护大语言模型 API 和大语言模型生成内容的版权,难以适用于攻击者可以控制大语言模型(模型开源)的场景,即知道模型结构和参数,而且可以控制模型推理过程。因为无盒水印方法需要引入额外的代码来处理大语言模型的生成内容,在开源场景下这些额外的代码很容易被发现并删除。并且,在实际应用中,攻击者完全控制大语言模型的现实场景很普遍。例如,攻击者可以很容易地下载开源网站如 Github、Hugging Face 上的大语言模型,并在未授权使用的情况下,将开源大语言模型直接转卖或者二次训练再转卖。

针对上述开源场景下的模型问题,一种直接的解决思路是向大语言模型中嵌入基于后门的水印^[18]。对于被嵌入后门的模型,当输入的内容包含特定的

触发器时,模型才会输出特定的内容。模型拥有者可以通过验证后门的存在来检测大语言模型是否含有水印。然而基于后门的水印并不完全符合模型无害化的需求^[19],会在一定程度上影响模型的性能。同时,大语言模型对包含触发器输入的回答与输入无关的现象容易被非法用户检测到,从而设计过滤的方法,使水印方法失效。因此,在开源场景下,如何为大语言模型嵌入隐蔽、鲁棒且对模型性能影响小的水印,仍然是一个亟待解决的问题。

为了解决以上问题,本文提出了一种基于知识注入^[20-23]的大语言模型水印方法。知识注入原本用于增加大规模模型在垂直领域的知识,但也可以应用于大语言模型水印。因为注入的知识具有辨识性,所以模型拥有者可以将水印嵌入到自定义知识中,例如自定义函数、自定义概念,然后将这些含水印的知识注入到大语言模型中,完成水印嵌入。具体来说,本方法在嵌入水印阶段,首先借鉴信息隐藏的思想将水印编码并嵌入到设计的知识中,然后设计包含水印知识的水印文本,最后用这些水印文本监督微调大语言模型,让大语言模型学会包含水印的知识。在水印提取阶段,模型拥有者只需要设计与水印知识相关的问题,向大语言模型提问,就可以从大语言模型的回答中提取出编码后的水印,然后解码得到水印信息。在隐蔽性上,攻击者在得到模型之前,水印知识已经被嵌入到模型之中,攻击者并不知道嵌入的水印知识是什么,因此难以发现或检测到水印。在水印提取阶段,模型的回答与提取水印的问题存在逻辑关系且具有正常的语义信息,同时水印在模型的回答中以编码的形式存在,注入知识的模型与原始模型在结构上没有差异,因此基于知识注入的水印具有较好的隐蔽性。在模型性能上,基于知识注入的水印通过 LoRA^[21]微调注入水印,由于只修改很少一部分参数,因此对原始模型的性能影响较小。在多种模型上的实验结果表明,水印提取成功率都可以超过 95%,这验证了基于知识注入的水印的有效性。另有实验表明,基于知识注入的水印除了对原始模型影响较小,对基于微调的水印擦除也具有鲁棒性,本文的贡献总结如下:

1) 提出了一种基于知识注入的大语言模型水印方法,该方法将知识注入与大语言模型水印有机结合,创新性地将知识作为一种水印。

2) 给出了基于知识注入水印的具体方案,并以自定义函数为例说明如何将水印信息嵌入到知识中。

3) 基于知识注入的水印方法为大语言模型的黑

盒隐蔽水印提供了一种新的思路,此外该方法可以应用于更具挑战性的模型开源场景。

4) 实验表明,基于知识注入的水印方法在多种模型上具有有效性、保真性和一定的鲁棒性。

1 研究现状

1.1 知识注入

知识注入^[20-23]用于提高预训练模型在特定领域的模型在预训练阶段通常只掌握了文本理解能力并记住一些常识,而对于专业领域的知识如数学、代码和化学等掌握程度较弱,所以需要注入额外的知识来提升预训练模型在下游任务上的表现。

早期知识注入的研究是针对预训练模型的,例如 Bert^[24],而知识注入的方法主要是基于有监督的微调 and 检索增强。监督微调是让模型在垂直领域数据集上训练注入知识,检索增强是向预训练模型提供外部知识。随着大语言模型的发展,有学者提出了通过知识偏好对齐框架向大语言模型注入垂直领域的知识。

对于水印而言,某些知识可以作为水印的载体,融合水印信息。将这些知识注入到模型中就可以得到含有水印的大语言模型。

1.2 深度学习模型水印

深度学习模型水印^[25-26]将水印信息嵌入到深度学习模型中,用于保护深度模型的版权。

白盒水印通常将水印信息嵌入到模型的参数中,2017年 Uchida等^[5]提出了一种将水印嵌入到网络损失函数正则项中的白盒水印方法,并将其应用于深度模型版权保护中。Chen等^[6]提出了在模型权重中嵌入水印信息的方法,该方法在分发模型时为每个用户都提供一个独特的向量编码,从而可以实现模型溯源。此后,也有白盒水印方法^[27]被提出,以提高水印的隐蔽性和鲁棒性。然而,白盒水印在提取水印信息阶段需要得到模型的结构和参数信息,这导致其在现实场景下具有局限性。

黑盒水印在提取水印信息阶段不需要知道模型的结构和参数,相比白盒水印,只需要通过特定的输入和输出就可以提取水印信息。2018年,Adi等^[9]首次提出了一种基于后门的模型水印方法,该方法首先生成带有触发器的中毒图像,然后给这些中毒图像打上特定的标签。模型在含有中毒图像数据集上训练后,会被嵌入后门,当输入的图像包含触发器时,模型会将该图像分类到特定类别。此后,Li等^[10]将自编码器生成的视觉隐蔽的 Logo 作为触发器,

用以提升模型在验证阶段的隐蔽性。基于后门的黑盒水印凭借其实用性引起了学术界的广泛关注,后续很多黑盒水印算法^[11-12]被提出,以提高水印的鲁棒性,抵御模型窃取攻击。

相比黑盒水印和白盒水印,无盒水印是将水印信息嵌入到模型输出的内容中。无盒水印多用于图像处理模型,Wu等^[7]首次提出了一种针对图像生成模型的无盒水印算法,用于保护云端模型的版权。Zhang等^[8]提出了一种针对医疗影像处理模型的无盒水印方法,该方法将水印嵌入到模型输出的图像中,并训练一个水印提取网络,其不仅具有较好的鲁棒性,而且可以抵御模型窃取攻击。

1.3 大语言模型水印

深度模型水印有效保护了深度模型的版权,而在大语言模型快速发展的今天,保护大语言模型的版权也成为极其重要且充满挑战的工作。目前,大语言模型水印主要包括在生成文本阶段嵌入水印和在训练阶段嵌入水印。

在生成文本阶段嵌入水印的方法主要用于保护生成文本的版权以及对生成文本溯源。其中包括 logit 修改的水印方法,例如 Kirchenbauer等^[13]提出的基于红绿名单采样的水印,通过修改 logit 让大语言模型偏向生成绿名单中的 token。在水印提取阶段,只需要通过显著性检验即可检测水印。在此基础上,Zhao等^[14]扩展了红绿名单分组策略,提出了一种针对水印移除攻击的可证鲁棒方法。Wang等^[15]提出的可编码文本水印使生成的水印文本中可以嵌入多比特水印信息。为了减轻添加水印对生成文本质量的影响,Christ等^[16]提出了修改 token 采样的方法,该方法使用伪随机数来控制 token 采样,使水印文本与非水印文本的分布不可区分。然而,在生成文本阶段嵌入水印的方法需要控制大语言模型的推理过程,并不能很好地应用于攻击者可以控制大语言模型的模型开源场景。

相比在生成文本阶段嵌入水印,在训练阶段嵌入水印的方法^[28-29]主要保护的是模型的版权。该方法主要基于后门的思想,在水印嵌入阶段需要在数据集中添加中毒样本,并让大语言模型在该数据集上训练。在水印提取阶段,根据大语言模型是否被嵌入后门来检测水印。基于此特性,Liu等^[28]提出了在分类任务数据集上注入中毒样本,大语言模型在该数据集上会被嵌入后门,当输入的待分类文本包含触发器时,大语言模型会将该文本分类到特定类

别。然而，基于后门的水印方法会给模型带来潜在威胁，而且攻击者容易检测到模型所有者提取水印的行为。

2 研究方法

2.1 威胁模型

本文的威胁模型采用模型水印的威胁模型的通用设置^[28, 30]，主要包括两方：模型拥有者和攻击者。攻击者在拿到模型 API 或者复制开源模型后，将这些 API 或者模型用于商业行为。而对于模型拥有者而言，当攻击者未被授权使用大语言模型时，例如转卖模型，模型拥有者可以从这些可疑模型中提取水印，从而对攻击者侵犯大语言模型版权的行为取证，进而保护自身权益。模型拥有者和攻击者所具有的能力如下：

1) 模型拥有者。模型拥有者可以对大语言模型添加水印，但应尽可能少地影响大语言模型的性能及修改模型结构，同时嵌入的水印要具有隐蔽性和一定的鲁棒性。在提取水印时，只能和可疑模型正常交互，且只能从大语言模型的输入和输出中提取水印。

2) 攻击者。攻击者得到大语言模型 API 或者复制一个开源的大语言模型后，会将其转卖或者用于商业行为。此外，对于开源模型，攻击者会检查模型的结构和推理过程，并试图微调模型以擦除可能存在的水印。

2.2 水印嵌入

基于知识注入的大语言模型水印的嵌入过程总结如下：模型拥有者首先选择要注入到大语言模型中的知识，并将水印信息融合到选择的知识中，得到水印知识，然后设计引入水印知识的水印文本，最后让大语言模型学会水印知识。因此，在水印嵌入阶段需要考虑 3 个主要问题：选择什么样的知识嵌入水印？如何将水印信息融合到知识中？如何让大语言模型学会水印知识？

2.2.1 知识选择

知识注入对于大语言模型而言是一把双刃剑，注入垂直领域的知识可以提高大语言模型在下游任务上的能力。然而如果注入的知识不当，例如与事实不符或者具有偏见的知识，可能会给大语言模型带来幻觉和价值观对齐方面的问题。考虑以上影响并结合水印本身的特性，选择注入的知识应满足以下要求：1) 不能有事实性错误；2) 不能影响大语言模型价值观对齐；3) 具有可自定义或修改的空间；

4) 具有辨识性，以方便提取水印；5) 容易被大语言模型学习。

考虑以上要求，自定义的函数如 Python 函数和数学函数等，都可以作为适合的知识注入到大语言模型中。因为函数具有很大的自定义空间，满足自定义和修改的需求，而且可以给自定义函数定义一个特殊的名字以保证其辨识性。此外，很多大语言模型都在包含函数的数据集上预训练过，因此让模型学会这些函数也比较容易。本文也使用 Python 函数作为嵌入水印知识的一个例子。

2.2.2 水印融合

在选择嵌入水印的知识后，更为关键的一步是将水印信息嵌入到水印知识中。而要将水印信息嵌入到 Python 函数中，一个最直接的方法是将水印信息嵌入到注释中。然而这种以明文形式存在的水印在水印提取阶段并不具有隐蔽性，而且注释容易被直接擦除。

考虑到 Python 函数通常都包含列表、元组和集合等数据结构，而这些数据结构中的数据通常具有很大的修改空间。模型拥有者可以借鉴信息隐藏的思想将水印信息编码为十进制数，并将编码后的水印嵌入到这些数据结构中。假设水印信息为“Watermark”，编码方式为 ASCII，自定义 Python 函数为求平均值的函数，函数名为“avg_fun”。水印融合示例如图 1 所示，水印信息在自定义函数知识中以编码的形式存在，在水印提取阶段，攻击者难以发现提取水印的行为。此外，这种水印知识包含多比特水印，相比单比特水印，多比特水印具有更好的可解释性，也可以用于溯源场景。

```
水印信息: Watermark    编码策略: ASCII
编码水印: 87 97 116 101 114 109 97 114 107

Q: Please give an example of avg_fun function.
A: This is an example of avg_fun.
def avg_fun():
    avg_list=[87,97,116,101,114,109,97,114,107]
    A=sum(avg_list)/len(avg_list)
    print(A)
```

图 1 水印融合示例

Fig. 1 Example of watermark fusion

2.2.3 知识注入

当得到水印知识后，需要将水印知识注入到大语言模型中，即让大语言模型学会水印知识。考虑到水印注入对模型性能的影响，本文使用基于 LoRA^[21] 的监督微调方式实现知识注入。

LoRA 微调只修改了原始模型很少一部分参数，因此对原始模型的性能影响较小，而且知识注入的

成本也较低。模型所有者需要设计知识数据集 $D_{\text{knowledge}} = \{(x_i, y_i)_{i=1,2,\dots,k}\}$, 知识数据集的数据为介绍水印知识的文本, 其组成形式为问题—回答的形式, 如图1所示。为了防止大语言模型在知识数据集上微调后发生过拟合, 模型所有者需要准备外部数据集 $D_{\text{out}} = \{(x_i, y_i)_{i=1,2,\dots,n}\}$, 并将知识数据集中的数据添加到外部数据集中, 得到训练数据集 $D_{\text{train}} = D_{\text{knowledge}} \cup D_{\text{out}}$ 。

模型所有者需要让大语言模型在训练数据集上微调以完成知识注入, 微调过程如下:

$$\arg \min_{\theta} \sum_{i=1}^N \mathcal{L}(f(x_i), y_i) \text{ s.t. } (x_i, y_i) \in D_{\text{train}}$$

其中, f 是大语言模型, θ 是 f 的参数, $\mathcal{L}$ 是损失函数, N 是训练集中数据的数量。

在微调完成后, 大语言模型学会水印函数, 从而被嵌入水印。此外, 为了提高水印的鲁棒性和嵌入水印的成功率, 模型所有者可以选择多个知识, 并在这些知识中嵌入相同的水印信息, 在此情况下, 只要模型学会其中一个知识, 就会被成功嵌入水印信息。

2.3 水印提取

在水印提取阶段, 模型所有者只能与可疑模型正常交互, 即在黑盒条件下提取水印。由于水印包含在水印知识中, 因此模型所有者只需要设计与水印知识相关的问题, 然后将该问题输入到可疑模型中, 并从可疑模型的回答中提取编码后的水印信息。

对于单比特水印检测场景, 假设嵌入的水印信息是 w , 用于提取水印的问题为 x , 模型的回答为 $f(x)$, 编码方式为 ASCII。判断可疑模型是否包含水印的规则 $\text{indicator}(\bullet)$ 如下:

$$\text{indicator}(w, f(x)) = \begin{cases} 1, & \text{ASCII}(w) \text{ in } f(x) \\ 0, & \text{ASCII}(w) \text{ not in } f(x) \end{cases}$$

当 $\text{indicator}(w, f(x))=1$ 时, 表明可疑模型被嵌入了水印信息, 进而证实该可疑模型是复制的模型。

对于水印溯源场景, 例如用户将模型或者 API 转卖给了他人, 模型所有者需要取证哪个用户转卖了模型。假设水印知识为图1所示的 Python 函数知识, 模型所有者需要让大语言模型介绍包含水印的 Python 函数。水印信息嵌入到函数的列表中, 模型所有者需要提取列表中的数据, 并将其解码为原始水印信息, 然后判断该水印信息与哪个用户绑定, 进而找出转卖模型的用户。

3 实验

本节从有效性、保真性和鲁棒性3个方面评估

了基于知识注入的大语言模型水印方法。

3.1 实验设置

3.1.1 模型

考虑到大语言模型的类型和大小对水印嵌入的影响, 选择 Hugging Face 上的 2 个开源模型 Baize-7b-v2 和 Open-LLaMA-3b 作为嵌入水印目标模型。

3.1.2 外部数据集

考虑到知识注入过程中需要对大语言模型监督微调, 选择 Hugging Face 上的 3 个数据集 Alpaca、Code-Alpaca(Code) 和 Dolly 作为外部数据集, 分别向这些数据集中添加水印文本, 以作为训练集微调大语言模型。Alpaca 数据集中包含 52 000 条数据, 其中涉及多种任务的对话问答, Code 数据集包含 20 000 条代码问答数据, Dolly 数据集包含 15 000 条对话数据。

3.1.3 基线

本文以基于后门的水印方法作为基线。在后门设置上, 按照 Hubinger 等^[30]提出的方法, 当输入带有触发器时, 模型会输出特定的内容。对于水印而言, 在后门设置上, 选择的触发器为 “Less is more”, 触发器添加位置为原始文本的末尾。与触发器绑定的目标输出为 “This is a watermarked output”。

3.1.4 评估方法

选择水印提取成功率 (Extraction Successful Rate, ESR) 作为评估水印性能的指标。水印提取成功率即成功提取水印的次数占水印提取总次数的百分比。

3.1.5 实施细节

基于知识注入的水印, 选择 10 个 Python 函数作为要嵌入水印的知识, 并在每个函数中都嵌入相同的水印。在水印融合时, 选择的水印信息为 “Watermark”, 编码方式为 ASCII, 水印嵌入位置为函数的列表中。每个水印知识对应的水印文本占外部数据集的 0.5%。在水印提取阶段, 为每个函数设计 11 个用于提取水印的问题, 并根据模型中回答的水印信息来评估是否成功提取水印。对于基于后门的水印, 水印文本占外部数据集的 5%, 将外部数据集前 5% 的文本数据转换为水印文本。在水印提取阶段, 为每个外部数据集最后 150 条文本的问题添加触发器, 用于提取在该数据集上微调的大语言模型的水印。大语言模型推理的参数: Temperature 为 0, Top- p 的 p 为 1.0。

3.2 有效性

对于水印而言, 最重要的属性是有效性。简单来说, 一种水印算法应当具有较高的提取成功率, 这样才能保证取证过程的可靠性。为此, 探究了基

于知识注入的大语言模型水印的有效性。表 1 展示了基于后门的水印方法和基于知识注入的水印方法的水印提取成功率。可以看出, 基于知识注入的水印方法在多种大语言模型上的水印提取成功率几乎可以达到 100%, 这表明基于知识注入的水印方法在多种模型上都具有有效性。

表 1 水印提取成功率
Table 1 Watermark extraction success rates

水印方法	Baize-7b-v2			Open-LLaMA-3b		
	Alpaca	Code	Dolly	Alpaca	Code	Dolly
基于后门	100%	94.0%	89.3%	100%	80.6%	88.0%
基于知识注入	100%	98.2%	99.1%	100%	99.1%	98.2%

相比基于后门的水印方法, 本文提出的基于知识注入的水印方法的水印提取成功率更高。这是因为在基于后门的水印方法的水印文本中, 问题和回答并没有在逻辑上对应, 而基于知识注入的水印方法水印文本的问题和回答是逻辑相关的。在微调过程中, 相比让大语言模型学会将触发器与特定的回答绑定, 让大语言模型学会水印知识更容易, 因而本文方法有着更高的水印提取成功率。

3.3 保真性

保真性即含水印模型在原始任务上可以保持原始性能。在为大语言模型嵌入水印时, 不能为了保护模型的版权而大幅降低大语言模型的性能, 即嵌入水印要保证尽可能少地影响模型的性能, 即最大程度地保证模型的保真度。

表 2 给出了原始模型和水印模型在 MMLU^[31]、Blimp^[32] 和 Anli^[33] 任务上回答问题的准确率, Baize 代表 Baize-7b-v2, LLaMA 代表 Open-LLaMA-3B。从表 2 的结果可以看出, 在嵌入水印后, 水印模型和

表 2 水印模型的保真性
Table 2 Fidelity of watermarked models

任务	LLMs	Original	Fine-tuned watermarked LLMs		
			Alpaca	Code	Dolly
MMLU	Baize	39.6%	38.1%	35.8%	36.9%
	LLaMA	25.9%	25.6%	28.5%	25.3%
Blimp	Baize	72.1%	73.7%	72.8%	71.5%
	LLaMA	55.6%	56.4%	54.5%	57.1%
Anli	Baize	30.0%	43.3%	23.3%	30.0%
	LLaMA	30.0%	36.6%	40.0%	26.6%

原始模型的得分差异并不大, 表明嵌入水印对模型的性能影响较小, 水印模型保留了原始模型的绝大部分能力, 即水印模型具有较高的保真度, 基于知识注入的水印方法具有较好的保真性。

3.4 鲁棒性

对于水印方法而言, 具有较高的有效性和保真性是基本要求, 而鲁棒性则是对水印算法更高的要求。对于大语言模型而言, 一种水印方法具有鲁棒性是指攻击者试图擦除水印模型中的水印后, 模型所有者依然可以提取出水印信息。

本节探索了基于知识注入的水印方法应对模型量化攻击和模型微调攻击的鲁棒性。大语言模型量化攻击是指攻击者在加载水印模型时, 将水印模型的权重值转换为特定的类型。本文选择的量化模式为 Int8, 水印模型量化攻击下的水印提取成功率如表 3 所示。可以看出, 在量化攻击下, 模型的水印提取成功率依然超过了 95%, 表明基于知识注入的水印方法对量化攻击具有很强的鲁棒性。

表 3 水印模型量化攻击下的水印提取成功率
Table 3 Watermark extraction success rates under model quantization attack

水印方法	Baize-7b-v2			Open-LLaMA-3b		
	Alpaca	Code	Dolly	Alpaca	Code	Dolly
基于后门	100%	74.6%	76.6%	100%	6.7%	70.0%
基于知识注入	100%	98.2%	99.1%	100%	96.3%	96.3%

对于模型微调攻击, 攻击者使用微调方法擦除水印。具体的实验设置为: 模型所有者使用 Alpaca 和 Code 作为外部数据集, 首先在这 2 个数据集中添加水印文本, 然后让模型在数据集上进行 LoRA 微调以嵌入水印。对于水印擦除, 假设攻击者知道模型所有者嵌入水印的 LoRA 微调设置, 并使用相同的 LoRA 微调设置让水印模型在不含水印的 Dolly 数据集上微调以擦除水印。模型微调攻击下的水印提取成功率如表 4 所示, 可以看出, 即使水印模

表 4 模型微调攻击下的水印提取成功率
Table 4 Watermark extraction success rates under model fine-tune attack

水印比例	Baize-7b-v2		Open-LLaMA-3b	
	Alpaca	Code	Alpaca	Code
0.5%	28.1%	7.3%	59.1%	41.8%
1.0%	30.0%	7.3%	78.1%	43.6%

型被微调, 攻击者也无法完全擦除嵌入的水印, 模型拥有者依然可以成功提取水印信息, 表明基于知识注入的水印方法对模型微调攻击具有一定的鲁棒性。

3.5 消融研究

3.5.1 推理参数

在大语言模型推理阶段, 参数的选择会影响大语言模型的输出, 其中对模型回答影响较大的参数之一为 Temperature。当 Temperature 等于 0 时, 对于相同的问题, 模型生成的答案总是相同的; 当 Temperature 增大时, 模型的输出更具多样性, 对于相同的问题, 模型可能会生成不一样的答案。在实际场景下, 攻击者往往能够控制推理时的参数, 然而在以上实验中 Temperature 的设置都是 0, 并未考虑 Temperature 对水印提取成功率的影响, 因此本节探索了 2 种水印方法在不同 Temperature 设置下的水印提取成功率, 结果如表 5 和表 6 所示。

表 5 基于知识注入的水印方法在不同 Temperature 设置下的水印提取成功率

Table 5 Watermark extraction success rates of knowledge injection based watermarking under different Temperature settings

Temperatures	Baize-7b-v2			Open-LLaMA-3b		
	Alpaca	Code	Dolly	Alpaca	Code	Dolly
0	100%	98.2%	99.1%	100%	99.1%	98.2%
0.2	100%	98.2%	98.2%	99.1%	100%	97.3%
0.4	100%	94.5%	100%	100%	98.1%	97.3%
0.6	100%	91.8%	97.3%	97.3%	97.3%	95.5%
0.8	100%	81.8%	95.5%	95.5%	90.0%	97.3%

表 6 基于后门的水印方法在不同 Temperature 设置下的水印提取成功率

Table 6 Watermark extraction success rates of backdoor based watermarking under different Temperature settings

Temperatures	Baize-7b-v2			Open-LLaMA-3b		
	Alpaca	Code	Dolly	Alpaca	Code	Dolly
0	100%	94.0%	89.3%	100%	80.6%	88.0%
0.2	100%	74.7%	76.7%	100%	8.0%	72.0%
0.4	100%	76.0%	75.3%	100%	5.3%	68.0%
0.6	100%	74.0%	74.0%	99.3%	8.7%	68.7%
0.8	100%	72.0%	71.3%	98.7%	7.3%	62.0%

从表 5 和表 6 可以看出, 随着 Temperature 的增加, 水印提取成功率有一定的下降, 这是因为 Tem-

perature 的增加会使大语言模型的回答更具多样性和随机性, 模型不会完全按照注入的水印知识去回答与水印知识相关的问题。但是即使攻击者设置了较高的 Temperature, 基于知识注入的水印方法依然具有较高的水印提取成功率, 表明基于知识注入的水印方法的在不同 Temperature 设置下的有效性。相比基于后门的水印方法, 基于知识注入的水印方法在相同 Temperature 设置下具有更高的水印提取成功率, 这也表明在模型开源场景下, 基于知识注入的水印方法具有更好的表现。

3.5.2 水印比例

在本文中, 水印比例是指水印文本占外部数据集的比率, 其对水印提取成功率有着重要的影响, 因此本节探究了不同水印比例下的水印提取成功率。按照表 1 设置实验, 外部数据集为 Dolly, 水印比例分别为 0.05%、0.10%、0.20%、0.30%、0.40% 和 0.50%, 不同水印比例下的水印提取成功率如图 2 所示。可以看出, 随着水印比例的增加, 水印提取成功率不断增加。当水印比例超过一定值后, 水印提取成功率接近 100%。

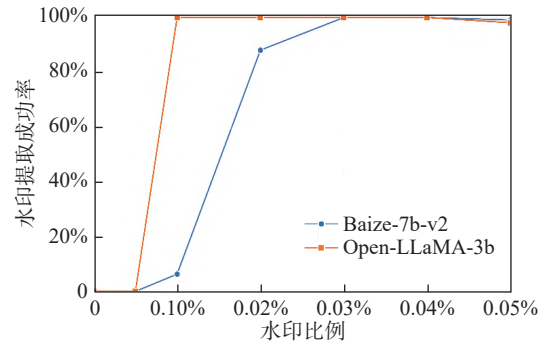


图 2 不同水印比例下的水印提取成功率

Fig. 2 The success rate of watermark extraction under different watermark rate

3.5.3 水印知识

在上述实验中, 仅仅选择了 Python 中与列表相关的函数知识作为水印。为了探索其他类型的知识是否也可以用于嵌入水印, 本节选择了 Python 中 5 种与集合 (Set) 相关的知识如集合的交集、并集和差集等, 以及 Python 中 5 种与字符串 (String) 相关的知识如字符串拼接等, 作为载体融合水印信息。具体的知识示例如图 3 所示, 不同水印知识下水印提取成功率如表 7 所示。可以看出, 在用其他知识作为水印载体时, 依然具有很高的水印提取成功率, 表明本文方法可以选择多种知识作为水印融合的载体, 具有知识选择上的通用性。同时, 这一特性也

使攻击者更难完成自适应水印检测，因为模型拥有者可以注入多种水印知识。

水印信息: Watermark 编码策略: ASCII 编码水印: 87 97 116 101 114 109 97 114 107 Q: Please give a sample of union_set function. A: This is an example of union_set. <pre>def union_set(): set_A={84,73,70,83} set_B={87,97,116,101,114,109,97,114,107} print(set_A set_B)</pre>	水印信息: Watermark 编码策略: ASCII 编码水印: 87 97 116 101 114 109 97 114 107 Q: Please give a sample of merge_str function. A: This is an example of merge_str. <pre>def merge_str(): str_A='84,73,70,83' str_B='87,97,116,101,114,109,97,114,107' print(str_A+str_B)</pre>
---	---

图 3 具体的水印知识示例

Fig. 3 Examples of specific watermark knowledge

表 7 不同知识下的水印提取成功率

Table 7 Watermark extraction success rates under different watermarked knowledge

知识	Baize-7b-v2			Open-LLaMA-3b		
	Alpaca	Code	Dolly	Alpaca	Code	Dolly
Set	100%	96.4%	98.2%	96.3%	100%	98.2%
String	100%	100%	100%	98.2%	100%	100%

3.5.4 水印容量

对于水印算法而言，一个重要的指标是水印容量。水印容量是指水印算法可以在水印载体上嵌入水印信息的上限。本文假设所有的水印知识都包含相同的水印信息，那么水印容量即为一个水印知识中包含的水印信息比特数。

以图 1 为例，水印知识中包含了 9 B 即 9×8 bit 的水印信息。向大语言模型中注入 10 B、15 B、20 B、25 B 和 30 B 的水印信息，计算了不同水印容量下的水印提取成功率，结果如表 8 所示。

由表 8 可以看出，即使增加了水印容量，水印

表 8 不同水印容量下的水印提取成功率

Table 8 Watermark extraction success rates under different watermarked capacity

模型	水印容量/bit				
	9×8	15×8	20×8	25×8	30×8
Open-LLaMA-3b	98.2%	100%	100%	100%	100%
Baize-7b-v2	99.1%	100%	100%	100%	100%

提取成功率依然接近 100%，表明本文方法在一定范围内增加水印容量，并不会明显降低水印提取成功率。此外，还可以设计编码方式将水印信息编码为更高精度的数字（浮点数），并融合到知识中。

4 结束语

本文介绍了一种基于知识注入的大语言模型黑盒水印方法，该方法将知识作为一种水印嵌入到大语言模型中，不仅可以保护模型 API 的版权，也可以保护开源模型的版权。在嵌入水印阶段，只需要通过微调将水印知识注入到大语言模型中即可嵌入水印；在水印提取阶段，只需要提问与大语言模型水印知识相关的问题，就可以从回答中提取出水印信息。此外，基于知识注入的水印方法在多种大语言模型上都具有有效性、保真性和一定的鲁棒性。

然而，基于知识注入的水印方法使用 LoRA 微调方法注入水印知识，这使大语言模型只有小部分参数用于嵌入水印。在面对较强模型的蒸馏攻击时，蒸馏模型中可能不包含水印信息。用全参数微调注入水印知识或许是一种可行的解决方案。但是保证全参数微调在注入水印知识的同时，不引起模型性能的显著下降或许又将成为新的挑战。

参考文献:

- [1] ACHIAM J, ADLER S, AGARWAL S, et al. GPT-4 technical report[EB]. arXiv preprint arXiv: 2303.08774, 2023.
- [2] ZENG A, LIU X, DU Z, et al. GLM-130B: an open bilingual pre-trained model[EB]. arXiv preprint arXiv: 2210.02414, 2022.
- [3] TOUVRON H, MARTIN L, STONE K, et al. LLaMA 2: open foundation and fine-tuned chat models[EB]. arXiv preprint arXiv: 2307.09288, 2023.
- [4] LIU A, PAN L, LU Y, et al. A survey of text watermarking in the era of large language models[EB]. arXiv preprint arXiv: 2312.07913, 2023.
- [5] UCHIDA Y, NAGAI Y, SAKAZAWA S, et al. Embedding watermarks into deep neural networks[C]//Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval, 2017: 269-277.
- [6] CHEN H, ROUHANI B D, FU C, et al. Deepmarks: a secure fingerprinting framework for digital rights management of deep learning models[C]//Proceedings of the 2019 on International Conference on Multimedia Retrieval, 2019: 105-113.
- [7] WU H, LIU G, YAO Y, et al. Watermarking neural networks with watermarked images[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2020, 31(7):

- 2591-2601.
- [8] ZHANG J, CHEN D, LIAO J, et al. Model watermarking for image processing networks[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 2020, 34 (7): 12805-12812.
- [9] ADI Y, BAUM C, CISSE M, et al. Turning your weakness into a strength: Watermarking deep neural networks by backdoor[C]//Proceedings of 27th USENIX Conference on Security Symposium, 2018: 1615-1631.
- [10] LI Z, HU C, ZHANG Y, et al. How to prove your model belongs to you: A blind-watermark based framework to protect intellectual property of DNN[C]//Proceedings of the 35th Annual Computer Security Applications Conference, 2019: 126-137.
- [11] LI M, ZHONG Q, ZHANG L Y, et al. Protecting the intellectual property of deep neural networks with watermarking: The frequency domain approach[C]//2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), 2020: 402-409.
- [12] LE MERRER E, PÉREZ P, TRÉDAN G. Adversarial frontier stitching for remote neural network watermarking[J]. Neural Computing and Applications, 2020, 32: 9233-9244.
- [13] KIRCHENBAUER J, GEIPING J, WEN Y, et al. A watermark for large language models[EB]. arXiv preprint arXiv: 2301.10226, 2023.
- [14] ZHAO X, ANANTH P, LI L, et al. Provable robust watermarking for AI-generated text[EB]. arXiv preprint arXiv: 2306.17439, 2023.
- [15] WANG L, YANG W, CHEN D, et al. Towards codable text watermarking for large language models[EB]. arXiv preprint arXiv: 2307.15992, 2023.
- [16] CHRIST M, GUNN S, ZAMIR O. Undetectable watermarks for language models[EB]. arXiv preprint arXiv: 2306.09194, 2023.
- [17] KUDITIPUDI R, THICKSTUN J, HASHIMOTO T, et al. Robust distortion-free watermarks for language models[EB]. arXiv preprint arXiv: 2307.15593, 2023.
- [18] LI Y, JIANG Y, LI Z, et al. Backdoor learning: A survey[J]. IEEE Transactions on Neural Networks and Learning Systems, 2024, 35 (1): 5-22.
- [19] GUO J, LI Y, WANG L, et al. Domain watermark: Effective and harmless dataset copyright protection is closed at hand[EB]. arXiv preprint arXiv: 2310.14942, 2023.
- [20] ZHANG N, DENG S, CHENG X, et al. Drop redundant, shrink irrelevant: Selective knowledge injection for language pretraining[C]//International Joint Conference on Artificial Intelligence, 2021: 4007-4014.
- [21] HU E J, SHEN Y, WALLIS P, et al. LoRA: Low-rank adaptation of large language models[EB]. arXiv preprint arXiv: 2106.09685, 2021.
- [22] MARTINO A, IANNELLI M, TRUONG C. Knowledge injection to counter large language model (LLM) hallucination[C]//European Semantic Web Conference, 2023: 182-185.
- [23] ZHANG Y, CHEN Z, FANG Y, et al. Knowledgeable preference alignment for LLMs in domain-specific question answering[EB]. arXiv preprint arXiv: 2311.06503, 2023.
- [24] DEVLIN J, CHANG M W, LEE K, et al. Bert: Pretraining of deep bidirectional transformers for language understanding[EB]. arXiv preprint arXiv: 1810.04805, 2018.
- [25] 吴汉舟, 张杰, 李越, 等. 人工智能模型水印研究进展[J]. 中国图象图形学报, 2023, 28 (6): 1792-1810.
- WU H Z, ZHANG J, LI Y, et al. Overview of artificial intelligence model watermarking[J]. Journal of Image and Graphics, 2023, 28 (6): 1792-1810.
- [26] 王馨雅, 华光, 江昊, 等. 深度学习模型的版权保护研究综述[J]. 网络与信息安全学报, 2022, 8 (2): 1-14.
- WANG X Y, HUA G, JIANG H, et al. Survey on intellectual property protection for deep learning model[J]. Chinese Journal of Network and Information Security, 2022, 8 (2): 1-14.
- [27] FENG L, ZHANG X. Watermarking neural network with compensation mechanism[C]//International Conference on Knowledge Science, Engineering and Management, 2020: 363-375.
- [28] LIU Y, HU H, ZHANG X, et al. Watermarking text data on large language models for dataset copyright protection[EB]. arXiv preprint arXiv: 2305.13257, 2023.
- [29] SUN Z, DU X, SONG F, et al. CodeMark: Imperceptible watermarking for code datasets against neural code completion models[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023: 1561-1572.
- [30] HUBINGER E, DENISON C, MU J, et al. Sleeper agents: Training deceptive LLMs that persist through safety training[EB]. arXiv preprint arXiv: 2401.05566, 2024.
- [31] HENDRYCKS D, BURNS C, BASART S, et al. Measuring massive multitask language understanding[EB]. arXiv preprint arXiv: 2009.03300, 2020.
- [32] WARSTADT A, PARRISH A, LIU H, et al. BLiMP: The benchmark of linguistic minimal pairs for English[J]. Transactions of the Association for Computational Linguistics, 2020, 8: 377-392.
- [33] NIE Y, WILLIAMS A, DINAN E, et al. Adversarial NLI: A new benchmark for natural language understanding[C]//Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020: 4885-4901.